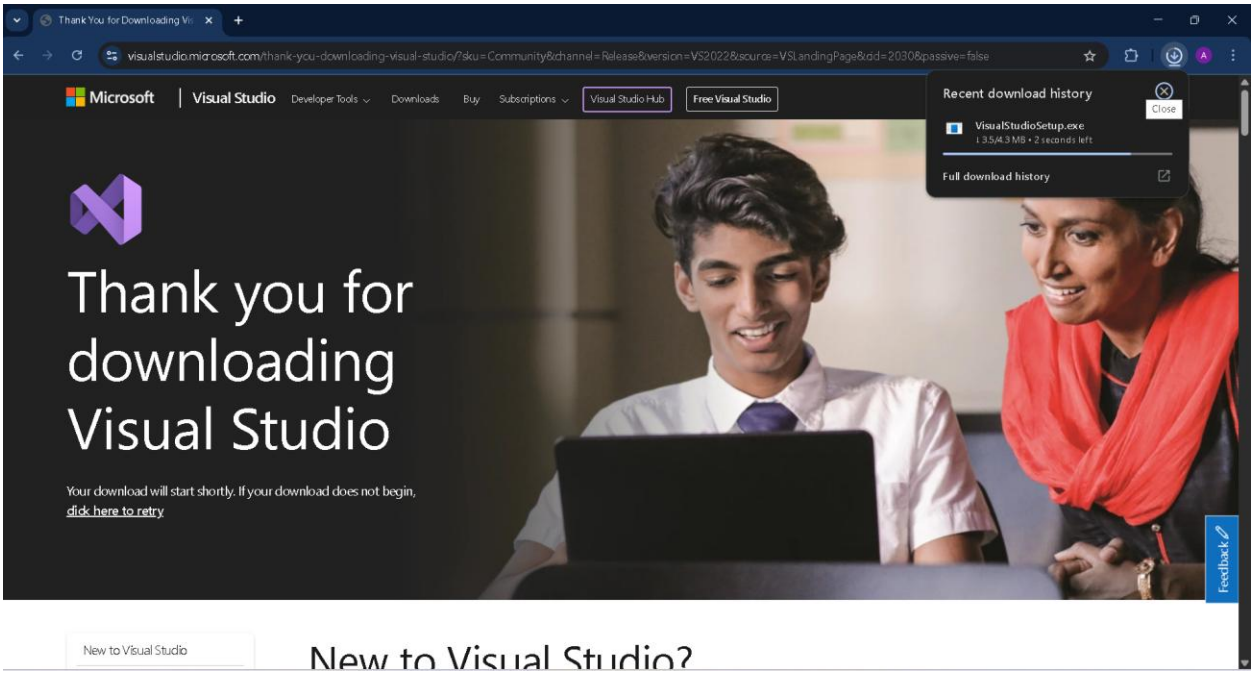
Lab no: 1 **Date:2082/03/04**

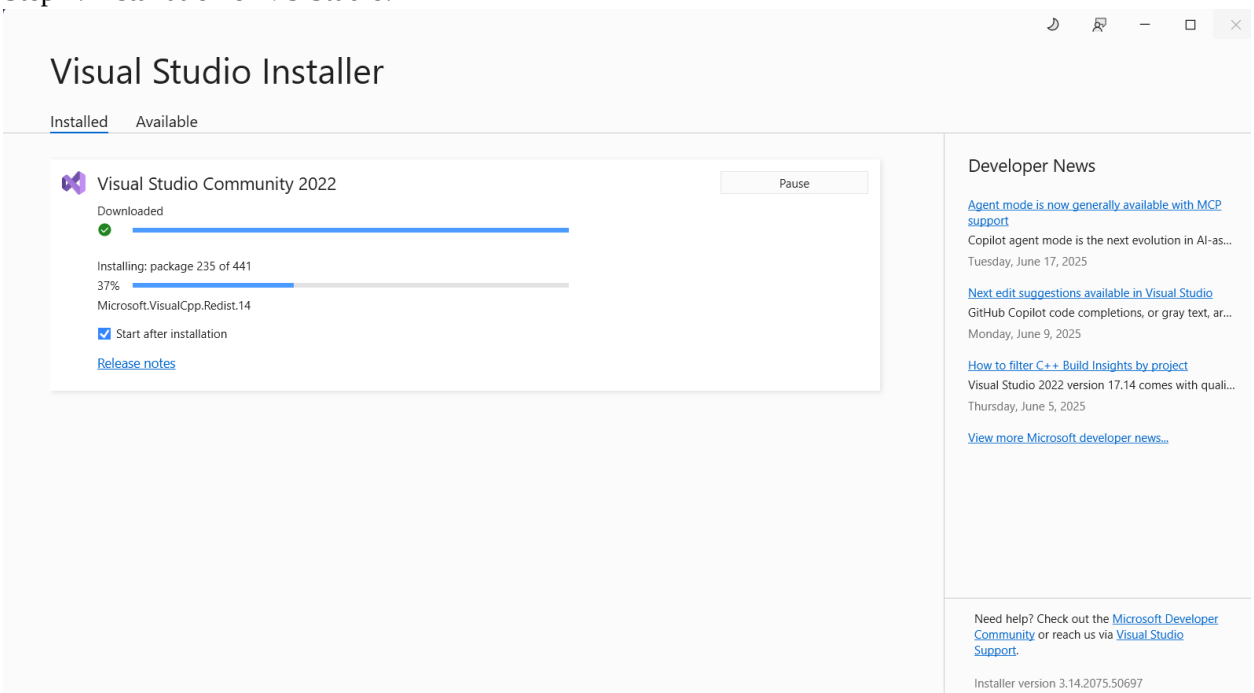
Title: Installation of Visual Studio.

Step 1: Download Visual Studio 2022

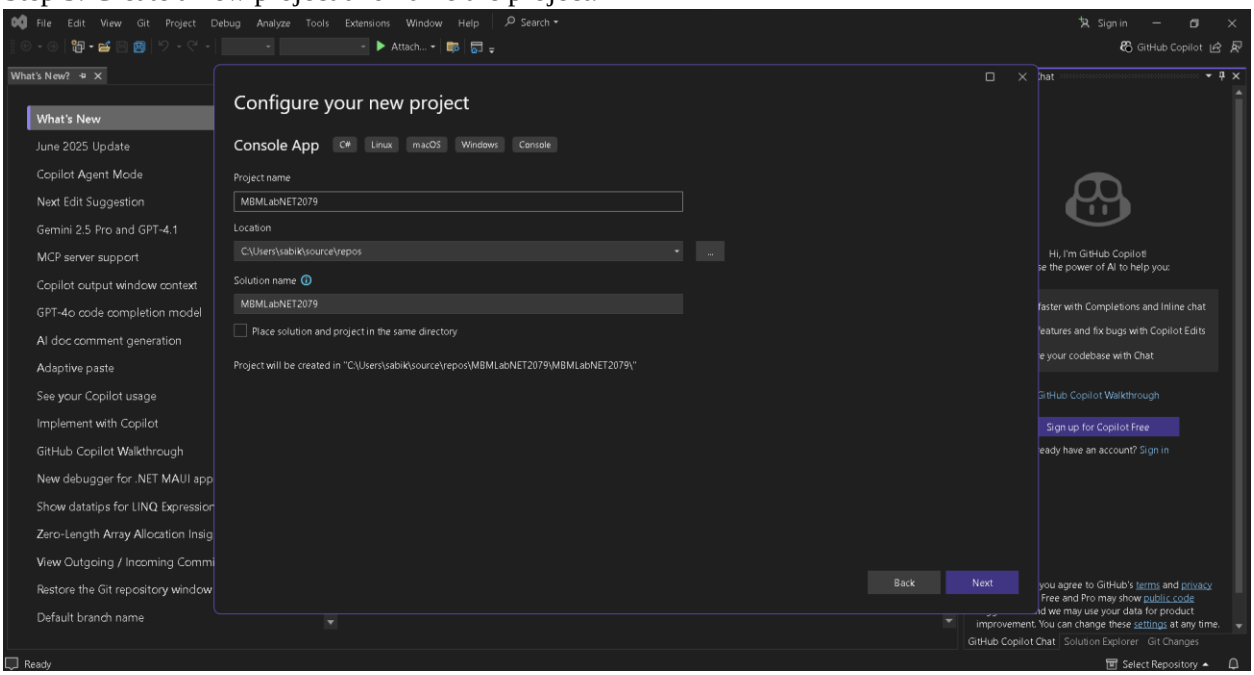
https://visualstudio.microsoft.com/vs



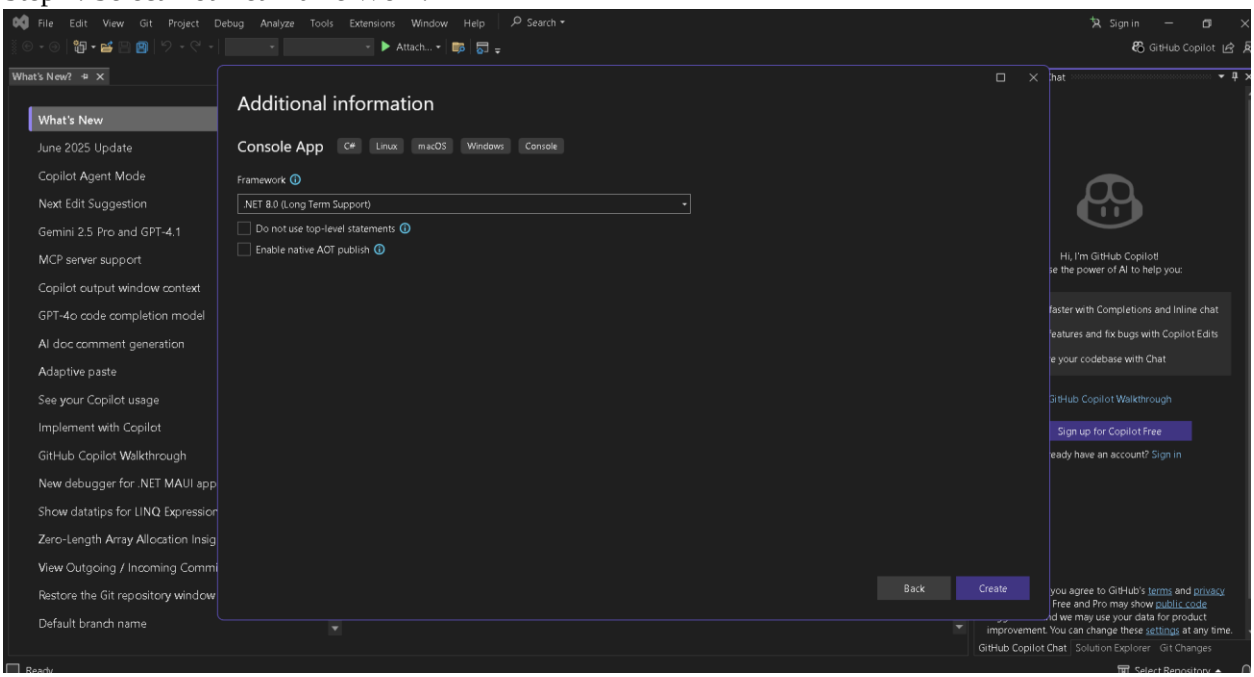
Step 2: Installation of VS Studio.



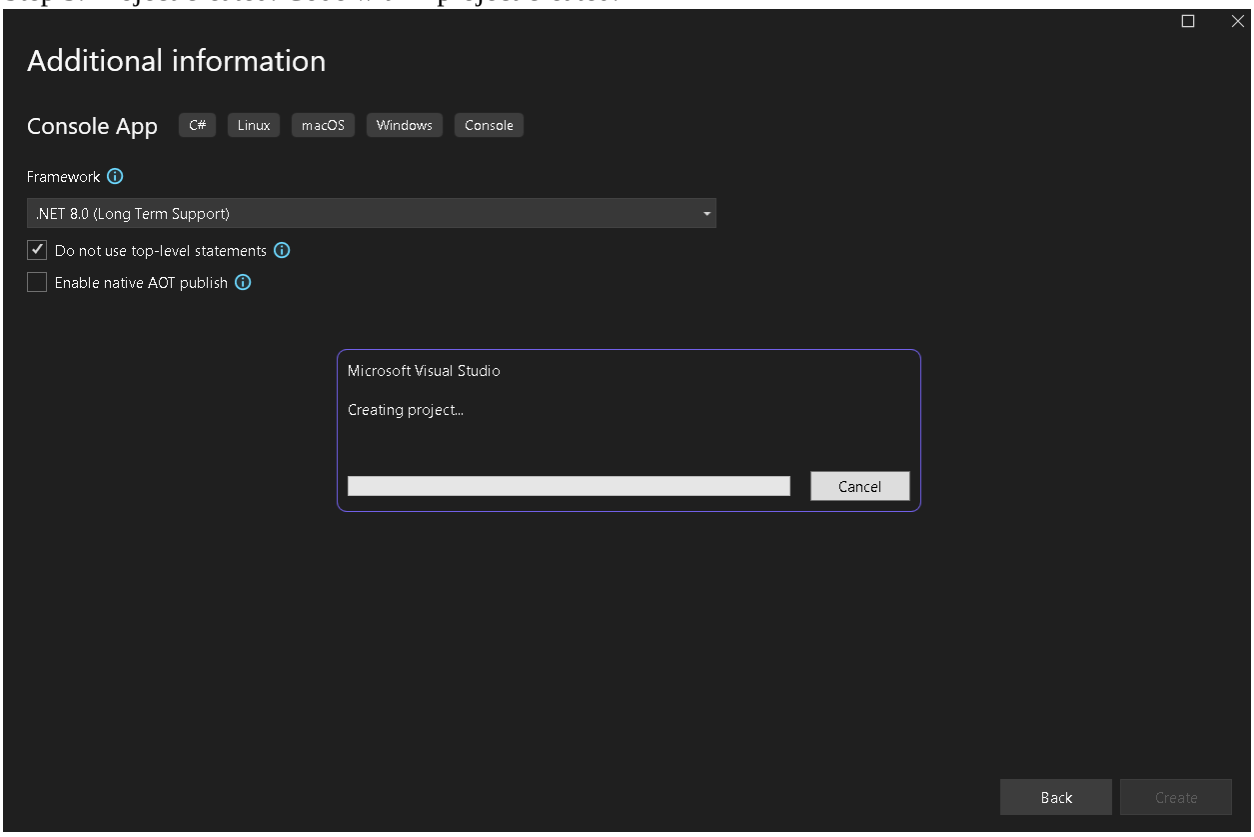
Step 3: Create a new project and name the project.



Step 4: Select Dot Net Frame Work.



Step 5: Project created. Code within project created.



Lab no: 2 **Date:2082/03/04**

Title: Introduction to C Sharp Program.

Theory:

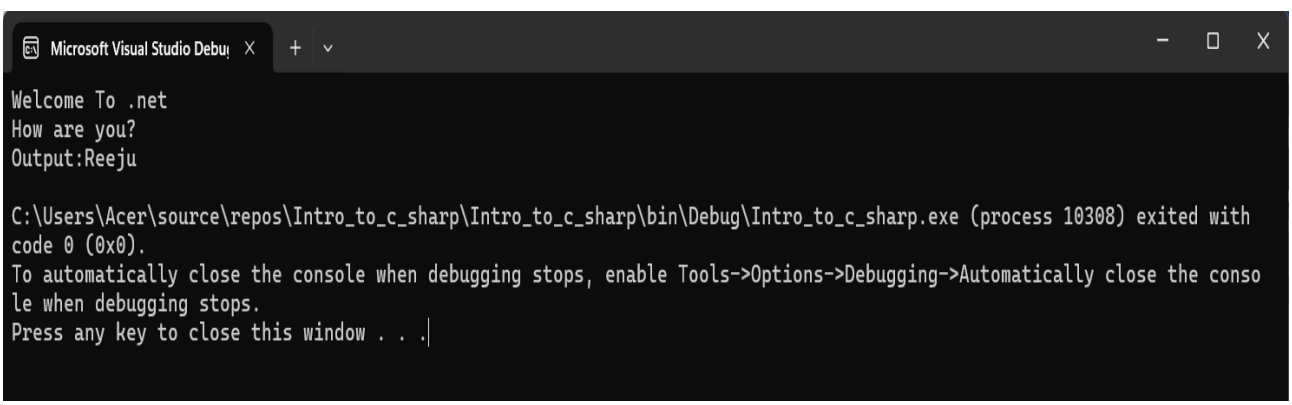
C# is a modern, object-oriented programming language developed by Microsoft around 2002 that runs on the .NET platform. It uses a clear, C-style syntax similar to Java and C++, and features strong static typing, automatic memory management (garbage collection), and advanced tools like LINQ and async /await.

Source Code:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Intro_to_c_sharp
{
    internal class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Welcome To .net");
            Console.WriteLine("How are you?");
            Console.WriteLine("Output:Reeju");
        }
    }
}
```

Outpu:



Lab no: 3 **Date:2082/03/04**

Title: Program to Add two numbers in C#.

Source Code:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Intro_to_c_sharp
{
    internal class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Enter First number: ");
            int num1 = int.Parse(Console.ReadLine());
            Console.WriteLine("Enter Second number: ");
            int num2 = int.Parse(Console.ReadLine());
            int sum = num1 + num2;
            Console.WriteLine($"The Sum is: {sum}");
            Console.WriteLine("Output:Reeju");
        }
    }
}
```

Output:

```
Microsoft Visual Studio Debu

Enter First number:
15
Enter Second number:
25
The Sum is: 40
Output:Reeju

C:\Users\Acer\source\repos\Intro_to_c_sharp\Intro_to_c_sharp\bin\Debug\Intro_to_c_sharp.exe (process 13700) exited with
code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the conso
le when debugging stops.
Press any key to close this window . . .
```

Source Code:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Intro_to_c_sharp
{
    internal class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Enter the First name:");
            string firstName = Console.ReadLine();
            Console.WriteLine("Enter the Last name:");
            string lastName = Console.ReadLine();
            string fullName = $"{firstName} {lastName}";
            Console.WriteLine("The Full Name is: " + fullName);
        }
    }
}
```

Output:

```
Microsoft Visual Studio Debu

Enter the First name:
Reeju
Enter the Last name:
Pandit
The Full Name is: Reeju Pandit

C:\Users\Acer\source\repos\Intro_to_c_sharp\Intro_to_c_sharp\bin\Debug\Intro_to_c_sharp.exe (process 11980) exited with
code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the conso
le when debugging stops.
Press any key to close this window . . .
```

Lab no: 4 **Date:2082/03/05**

Title: Implementation on Datatypes in C#.

Theory:

In C#, data types are used to define the kind of data a variable can hold. They are mainly divided into value types and reference types. Value types (like int, float, char, and bool) store data directly and are stored in the stack.

Source Code:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Datatype_In_C_
{
    internal class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine(int.MaxValue);
            Console.WriteLine(int.MinValue);

            Console.WriteLine(sbyte.MaxValue);
            Console.WriteLine(sbyte.MinValue);

            Console.WriteLine(byte.MaxValue);
            Console.WriteLine(byte.MinValue);

            Console.WriteLine(short.MaxValue);
            Console.WriteLine(short.MinValue);

            Console.WriteLine(ushort.MaxValue);
            Console.WriteLine(ushort.MinValue);

            Console.WriteLine(int.MaxValue);
            Console.WriteLine(int.MinValue);

            Console.WriteLine(uint.MaxValue);
            Console.WriteLine(uint.MinValue);

            Console.WriteLine(long.MaxValue);
            Console.WriteLine(long.MinValue);

            Console.WriteLine(ulong.MaxValue);
            Console.WriteLine(ulong.MinValue);

            Console.WriteLine(float.MaxValue);
            Console.WriteLine(float.MinValue);

            Console.WriteLine(double.MaxValue);
            Console.WriteLine(double.MinValue);

            Console.WriteLine(decimal.MaxValue);
            Console.WriteLine(decimal.MinValue);
```

```
            Console.WriteLine("\\u" + ((int)char.MinValue).ToString("X4"));
            Console.WriteLine("\\u" + ((int)char.MaxValue).ToString("X4"));
            Console.WriteLine("Output by:Reeju Pandit");
            Console.ReadLine();
        }
    }
}
```

Output:

```
C:\Users\Acer\source\repos\C

2147483647
-2147483648
Output by:Reeju Pandit
```

```
C:\Users\Acer\source\repos\C

127
-128
Output by:Reeju Pandit
```

```
C:\Users\Acer\source\repos\C

255
0
Output by:Reeju Pandit
```

```
C:\Users\Acer\source\repos\C

32767
-32768
Output by:Reeju Pandit
```

```
C:\Users\Acer\source\repos\C

65535
0
Output by:Reeju Pandit
```

```
C:\Users\Acer\source\repos\C

2147483647
-2147483648
Output by:Reeju Pandit
```

```
C:\Users\Acer\source\repos\C

4294967295
0
Output by:Reeju Pandit
```

```
C:\Users\Acer\source\repos\C

9223372036854775807
-9223372036854775808
Output by:Reeju Pandit
```

```
C:\Users\Acer\source\repos\C

3.402823E+38
-3.402823E+38
Output by:Reeju Pandit
```

```
C:\Users\Acer\source\repos\C

1.79769313486232E+308
-1.79769313486232E+308
Output by:Reeju Pandit
```

```
C:\Users\Acer\source\repos\C

79228162514264337593543950335
-79228162514264337593543950335
Output by:Reeju Pandit
```

```
C:\Users\Acer\source\repos\C

\u0000
\uFFFF
Output by:Reeju Pandit
```

Lab no: 5 **Date:2082/03/05**

Title: Implementation of Boolean Datatypes in C#.

Theory:

In C#, the bool type is simple built-in value type that can only be true or false, making it perfect for conditions, comparisons, and logical operations like &&, ||, and !. Its default value (when declared without initialization) is false, and although it only needs one bit, it occupies 1 byte in memory due to alignment rules.

Source Code:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Bool_Datatype
{
    internal class Program
    {
        static void Main(string[] args)
        {
            int x = 2;
            int y = 3;
            bool c = x > y;
            Console.WriteLine(c);
            Console.WriteLine("Reeju Pandit");
        }
    }
}
```

Output:

```
Microsoft Visual Studio Debu

False
Reeju Pandit

C:\Users\Acer\source\repos\Bool Datatype\Bool Datatype\bin\Debug\Bool Datatype.exe (process 6924) exited with code 0 (0x
0).
Press any key to close this window . . .
```

Lab no: 6 **Date:2082/03/05**

Title: Implementation of Float and Double Datatypes in C#.

Theory:

In C#, float and double are used to store decimal numbers with fractional parts. A float is a 32-bit single-precision type that provides about seven digits of accuracy and a range of approximately $\pm 3.4 \times 10^{38}$, making it suitable for memory-sensitive tasks like graphics. On the other hand, a double is a 64-bit double-precision type offering around 15 to 17 digits of accuracy and a much wider range. It is the default choice for most calculations because it provides better precision and performance. Generally, double is preferred unless memory or platform constraints specifically require the use of float.

Source Code:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Lab6_Float__and__Double_datatype
{
    internal class Program
    {
        static void Main(string[] args)
        {
            float a = 23.365f;
            double b = 23.3650847897387d;
            Console.WriteLine($"The value of a is: {a} and the value of b is: {b}");
            Console.WriteLine("Outout by Reeju Pandit");
        }

    }
}
```

Output:

```
Microsoft Visual Studio Debu

The value of a is: 23.365 and the value of b is: 23.3650847897387
Outout by Reeju Pandit

C:\Users\Acer\source\repos\Lab6_Float _and _Double_datatype\Lab6_Float _and _Double_datatype\bin\Debug\Lab6_Float _and _
Double_datatype.exe (process 15868) exited with code 0 (0x0).
Press any key to close this window . . .
```

Lab no: 7 **Date:2082/03/06**

Title: Implementation of String Datatype in C#.

Theory:

In C#, the string datatype is used to store a sequence of characters. It is an object of the System. String class and is immutable, meaning its value cannot be changed once created. String operations like concatenation, comparison, and substring can be done using built-in methods. Strings are commonly written inside double quotes, for example: string name = "Reeju";

Source Code:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace String_datatype7
{
    internal class Program
    {
        static void Main(string[] args)
        {
            //string a = "Welcome to .NET";
            //string a = "\"Welcome to .NET\"";
            //string a = "\\Welcome to C#\\";
            string a = "Welcome to C#\nTwo";
            Console.WriteLine(a);
            Console.WriteLine("Output By:Reeju Pandit");
            Console.ReadLine();

        }
    }
}
```

Output:

```
Microsoft Visual Studio Debu
Welcome to .NET
Output By:Reeju Pandit


C:\Users\Acer\source\repos\String_datatype7\String_datatype7\bin\Debug\String_datatype7.exe (process 10348) exited with
code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the conso
le when debugging stops.
Press any key to close this window . . .
```

```
Microsoft Visual Studio Debu
"Welcome to .NET"
Output By:Reeju Pandit


C:\Users\Acer\source\repos\String_datatype7\String_datatype7\bin\Debug\String_datatype7.exe (process 7368) exited with c
ode 0 (0x0).
Press any key to close this window . . .
```

```
\Welcome to C#\
Output By:Reeju Pandit


C:\Users\Acer\source\repos\String_datatype7\String_datatype7\bin\Debug\String_datatype7.exe (process 15140) exited with
code 0 (0x0).
Press any key to close this window . . .
```

```
Welcome to C#
Two
Output By:Reeju Pandit


C:\Users\Acer\source\repos\String_datatype7\String_datatype7\bin\Debug\String_datatype7.exe (process 15072) exited with
code 0 (0x0).
Press any key to close this window . . .
```

Lab no: 8 **Date:2082/03/06**

Title: Implementation of Verbatim String Datatype in C#.

Theory:

In C#, a verbatim string is defined using the @ symbol before the string literal. It allows the use of multi-line text and ignores escape sequences like \n or \\. This makes it useful for file paths or formatted text. For example:

string path = @"C:\Users\Reeju\Documents";

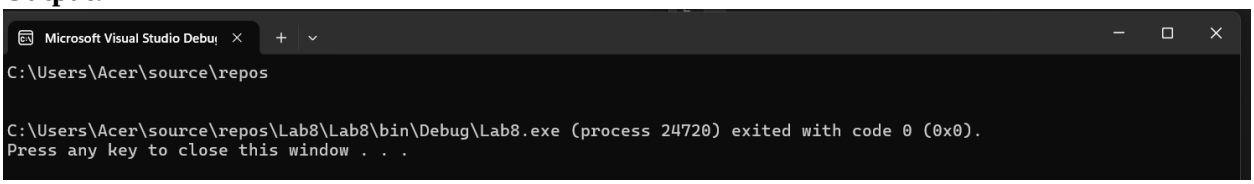
Verbatim strings improve readability and simplify handling of long or complex string values.

Source Code:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Lab8
{
    internal class Program
    {
        static void Main(string[] args)
        {
            //string x = "D:\\Adil\\Tutorial\\CSharp";;
            string x = @"C:\Users\Acer\source\repos";
            Console.WriteLine(x);
            Console.ReadLine();
        }
    }
}
```

Output:



Lab no: 9 **Date:2082/03/06**

Title: (a)Implementation of implicit conversion (int to float) in C#.

Theory:

In C#, implicit conversion happens automatically when the compiler knows there's no risk of data loss or error, so you don't need to write a cast. For example, converting a smaller numeric type like int to a larger one like long, or a float to a double, is implicit.

Source Code:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Lab9
{
    internal class Program
    {
        static void Main(string[] args)
        {
            int num = 10;
            float result = num;
            Console.WriteLine("Integer value: " + num);
            Console.WriteLine("Converted float value: " + result.ToString("0.0"));
        }
    }
}
```

Output:

```
Microsoft Visual Studio Debu

Integer value: 10
Converted float value: 10.0

C:\Users\Acer\source\repos\Lab9\Lab9\bin\Debug\Lab9.exe (process 10872) exited with code 0 (0x0).
Press any key to close this window . . .
```

Title: (b)Implementation of explicit conversion (float to int) in C#.

Theory:

In C#, explicit conversion (also known as type casting) lets you manually convert a value from one data type to another when the conversion might lose information or isn't automatically allowed. You write it as value. For instance, converting a double or float to an int requires you to use (int), because decimals are truncated and data can be lost. This makes your intent clear to the compiler and anyone reading your code.

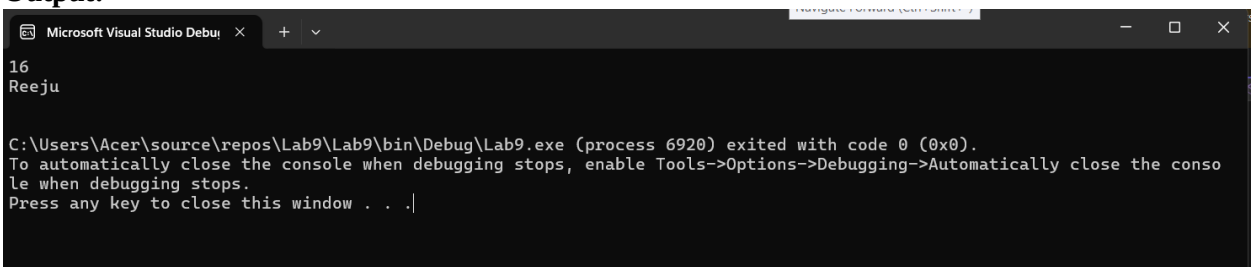
Source Code:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Lab9
{
    internal class Program
    {
        static void Main(string[] args)
        {
            float a = 16.234f;
            int b = (int)a;
            Console.WriteLine(b);
            Console.WriteLine("Reeju");
            Console.ReadLine();
        }

    }
}
```

Output:



Lab no: 10 **Date:2082/03/09**

Title: (a)Implementation of string to integer in C#.

Theory:

To convert a C# string to an int, you generally use one of three methods:

- **int.Parse(...) :**

Converts the string directly but throws an exception if the string is null or not a valid number.

- **Convert.ToInt32(...):**

Also converts strings and returns 0 for null, but still throws for invalid formats.

- **int.TryParse(..., out int result):**

The safest and most robust: it returns a bool indicating success or failure, never throws, and outputs the converted integer (or 0 if it fails).

Source Code:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Security.Cryptography;
using System.Text;
using System.Threading.Tasks;

namespace Lab10
{
    internal class Program
    {
        static void Main(string[] args)
        {
            string a = "52";
            string b = "68";
            int c=Convert.ToInt32(a)+Convert.ToInt32(b);
            Console.WriteLine(c);
            Console.WriteLine("Output:Reeju Pandit");
            Console.ReadLine();
        }
    }
}
```

Output:

```
Microsoft Visual Studio Debu
120
Output:Reeju Pandit


C:\Users\Acer\source\repos\Lab10\Lab10\bin\Debug\Lab10.exe (process 20848) exited with code 0 (0x0).
Press any key to close this window . . .
```

Title: (b)Implementation of string to float in C#.

Theory:

To convert a C# string to a float, you generally have three options:

- **float.Parse(...) :**

Directly converts the string but throws exceptions.

- **Convert.ToSingle(...):**

Similar to Parse, but returns 0 for null strings and still throws for invalid format.

- **float.TryParse(..., out float result):**

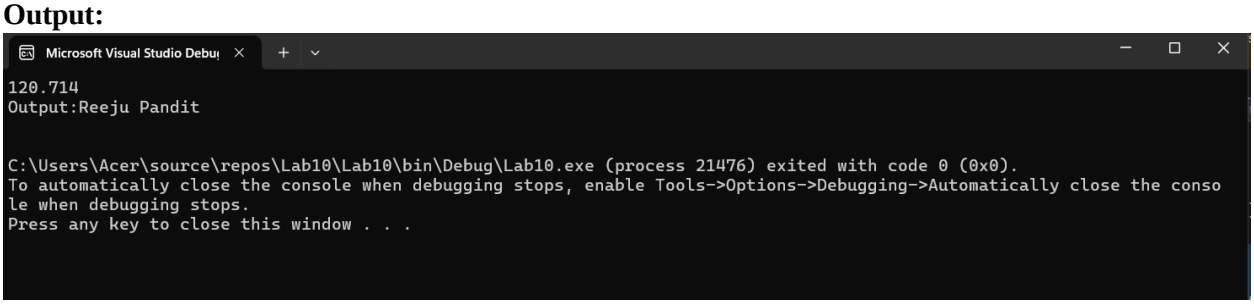
The safest option: it returns a bool indicating success, never throws, and outputs the converted value (or 0 if it fails).

Source Code:

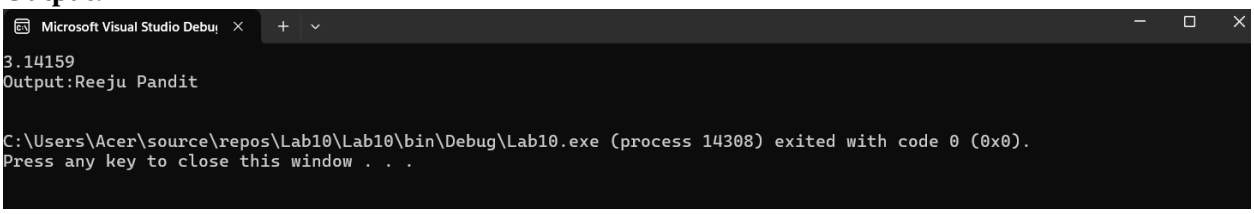
```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Security.Cryptography;
using System.Text;
using System.Threading.Tasks;

namespace Lab10
{
    internal class Program
    {
        static void Main(string[] args)
        {
             string a = "20.354";
              string b = "100.36";
             float c = float.Parse(a) + float.Parse(b);
             Console.WriteLine(c);
            Console.WriteLine("Output:Reeju Pandit");
            Console.ReadLine();
        }
    }
}
```

Output:



Title: (c)Implementation of constant in C#.

Theory:
In C#, a constant is a value that cannot be changed after it's set, defined using the const keyword. Constants must be assigned a value at the time of declaration and are implicitly static, meaning they belong to the type itself rather than an instance. They are evaluated at compile time, allowing for potential performance optimizations. Constants are commonly used for fixed values like mathematical constants (e.g., PI), configuration settings, or error codes.

Source Code:
```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Security.Cryptography;
using System.Text;
using System.Threading.Tasks;

namespace Lab10
{
    internal class Program
    {
        static void Main(string[] args)
        {
            const double PI = 3.14159;
            Console.WriteLine(PI);
            Console.WriteLine("Output:Reeju Pandit");
            Console.ReadLine();
        }
    }
}
```
Output: